

September 3, 2026

1 P.P1 Proyecto Final - Unidad 1

Arturo Vargas Espinosa - 611377

2 Introducción, Planteamiento del Problema y Contexto

En este reporte se construyen modelos de regresión para predecir el **ingreso mensual** (`ingocup`) de personas ocupadas en México, a partir de información sociodemográfica y laboral, usando los microdatos de la Encuesta Nacional de Ocupación y Empleo (ENOE) del primer trimestre de 2026, levantada por el INEGI. A diferencia de un conjunto de datos “de laboratorio”, esta base presenta los retos típicos de una encuesta real: códigos numéricos de no respuesta que pandas lee como valores válidos, variables que solo aplican a un subconjunto de la muestra, y variables que miden esencialmente lo mismo bajo otro nombre (colinealidad).

El interés detrás de este análisis es entender qué características de una persona y de su empleo están asociadas con su nivel de ingreso, y en qué medida. Factores como la escolaridad, las horas trabajadas, el sector de actividad o la posición dentro de la ocupación suelen mencionarse como determinantes del ingreso, pero rara vez se cuantifica su aporte individual de forma simultánea. Una regresión múltiple permite justamente eso: estimar el efecto de cada variable *manteniendo el resto constante*, en lugar de mirar cada factor de forma aislada. El objetivo no es solo lograr un modelo que prediga bien, sino documentar y justificar cada decisión tomada durante la limpieza, el análisis de relaciones entre variables y la selección de características, de forma que el proceso completo sea reproducible y quede claro por qué se llegó al modelo final.

Para este análisis se usará el archivo `enoe_2026_1t_limpio.csv`, resultado de unir dos de las cinco tablas que componen la ENOE (`SDEM`, sociodemográfico, y `COE2`, ocupación y empleo) mediante la llave primaria de ocho campos documentada por el INEGI, aplicando los filtros oficiales de calidad (entrevista completa, residentes habituales, población de 12 años y más) y conservando únicamente a quienes reportan ingreso por ocupación. El resultado son **126,226 observaciones**, cuyas variables se revisan a detalle en la sección de Exploración de los Datos.

2.1 Archivos

El archivo `enoe_2026_1t_limpio.csv` no es un dataset descargado directamente, sino el resultado de un proceso propio de integración. La ENOE distribuye la información en tablas separadas por cuestionario, y cada una por sí sola es insuficiente: `SDEM` (módulo sociodemográfico) trae las variables de la persona y del hogar, pero no el detalle de ocupación y empleo, mientras que `COE2` (segunda parte del cuestionario de ocupación y empleo) trae ese detalle pero no el resto del perfil sociodemográfico. Ninguna de las dos por separado permite construir el modelo, así que se combinan mediante la llave primaria de ocho campos que identifica de forma única a cada entrevistado (entidad, control, UPM, vivienda, selección, hogar, mudanza y renglón).

Antes de ese cruce, `SDEM` por sí sola trae **más de 400,000 registros**, pero la mayoría no sirve para este análisis: incluye entrevistas incompletas o no logradas, visitantes y trabajadores domésticos que no son residentes habituales del hogar, y personas fuera del rango de edad que cubre la encuesta (menores de 12 años, por ejemplo). Se aplican entonces tres filtros de calidad sobre `SDEM`: `r_def == 0` para quedarse solo con entrevistas completas, `c_res` en `{1, 3}` para conservar a los residentes habituales, y `eda` entre 12 y 98 años para respetar la población objetivo. El cruce con `COE2` se hace con un `inner join`, de modo que solo sobreviven los registros que existen en ambas tablas, y al final se descartan las observaciones sin `ingocup` positivo, ya que sin ingreso reportado no hay variable de respuesta que modelar. Después de estos filtros y del cruce, la base se reduce de los más de 400,000 registros iniciales a las **126,226 observaciones** con las que arranca el resto del reporte.

#Metodologia

Este estudio combina cuatro bloques del curso: **regresión lineal múltiple**, **solución de problemas de datos reales**, **selección de características** y **evaluación e inferencia**, más un modelo **no lineal** para comparar.

2.1.1 De la regresión lineal simple a la múltiple

La **regresión lineal simple** modela la relación entre una variable explicativa X y una respuesta Y como una recta:

$$Y = \beta_0 + \beta_1 X + \varepsilon$$

Los coeficientes se estiman por mínimos cuadrados, buscando la recta que minimiza $RSS = \sum (y_i - \hat{y}_i)^2$.

La **regresión lineal múltiple** generaliza el modelo a p variables explicativas:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_p X_p + \varepsilon$$

(Ecuaciones hechas con Claude)

La diferencia clave, y la razón por la que este proyecto la necesita, es que ahora cada β_i mide el efecto de X_i sobre Y **controlando por el resto de las variables**. Es decir, β_{mujer} ya no es la diferencia bruta de ingreso entre hombres y mujeres: es la diferencia que queda después de igualar escolaridad, horas trabajadas, sector, formalidad y todo lo demás que esté en el modelo.

Para saber si un coeficiente es real o podría deberse al azar se calcula $t = \beta_i / SE(\beta_i)$ y su p -value. El intervalo de confianza al 95% ($\beta_i \pm 1.96 \cdot SE(\beta_i)$) es el **margen de error** que se reporta en la sección de inferencia.

2.1.2 Modelo no lineal: bosque aleatorio

El modelo lineal impone una forma rígida: asume que el efecto de cada variable es constante y aditivo. En el mercado laboral eso es dudoso, porque hay efectos que claramente no son lineales (el ingreso sube con la edad hasta cierto punto y después baja) e interacciones (un año más de escolaridad probablemente vale más en el sector formal que en el informal).

Un **árbol de regresión** parte el espacio de las variables en regiones mediante cortes sucesivos y predice el promedio de Y dentro de cada región. Por construcción captura no linealidades e interacciones sin que se las especifiquemos. Su problema es que un árbol solo tiene mucha varianza.

El **bosque aleatorio** entrena muchos árboles sobre remuestreos *bootstrap* y, en cada corte, considera solo un subconjunto aleatorio de variables. Al promediar todos los árboles se reduce la varianza. El costo es que ya no hay coeficientes que leer, solo una medida de importancia relativa.

Se comparan los dos: el lineal para **explicar** y el bosque para ver **cuánta señal adicional existe** más allá de lo lineal.

Tabla 1. Tipos de variables

Tipo de variable	¿Qué es?	Ejemplo en este estudio
Cuantitativa	Numérica; representa una cantidad y admite operaciones aritméticas	eda (edad), anios_esc (años de escolaridad), hrsocup (horas trabajadas)
Cualitativa	Categorica; describe una clase o grupo, no una cantidad	sex , pos_ocu (posición en la ocupación), rama_est1 (sector económico)

#Exploración y comprensión del conjunto de datos

Empezamos importando pandas, numpy y matplotlib, y leyendo el archivo con `read_csv`.

Se imprimen las primeras observaciones con `df.head()` para ver con qué estamos trabajando.

```
[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

df = pd.read_csv('/content/enoe_2026_1t_limpio.csv', low_memory=False)

df.head(10)
```

```
[1]:   r_def  cve_ent   con   upm  n_pro_viv  v_sel  n_hog  h_mud  n_ren  \
0      0      9  40006  906681      117      5      1      0      1
1      0      9  40018  912598       63      3      1      0      5
2      0      9  40018  912598       81      4      1      0      3
3      0      9  40039  922769      109      5      1      0      2
4      0      9  40040  911404        7      2      1      0      1
5      0      9  40046  901679       89      4      1      0      1
6      0      9  40051  905367       90      3      1      0      1
7      0      9  40051  905367      136      5      1      0      1
8      0      9  40062  916613        4      1      1      0      2
9      0      9  40064  916869       76      1      1      0      1

   c_res  ...  tue1  pre_asa  niv_ins  anios_esc  hrsocup  ingocup  imssissste  \
0      1  ...      2        1        3        11        50    10000          1
1      1  ...      3        2        4        12        30    12900          4
2      1  ...      2        1        4        15        40    10000          2
3      1  ...      3        0        4        12        40    12900          4
4      1  ...      1        2        2         9        15     9000          4
5      1  ...      1        0        4        18        33   103200          4
6      1  ...      1        0        4        16        15    40000          4
7      1  ...      1        2        4        12        12     4300          4
8      1  ...      1        1        4        12        56    15050          1
9      1  ...      1        1        4        16        40    50000          1

   emp_ppal  p6c  p6d
0          2    1
1          1    6
2          2    3
3          1    6
4          1    6
5          2    6
6          2    6
7          1    6
8          2    1
9          2    1
```

[10 rows x 30 columns]

Este conjunto de datos contiene un total de 29 variables. El tipo de variable que más tiene este dataframe son numericas, siendo 27 variables tipo numericas y 1 variables categorica. Este tipo de variables es perfecta para proceder con nuestros modelos de regresión ya que la mayoría son numericas, aunque aun se necesitna evaluar los datos para eliminar colinealidad, outliers, nulls etc.

```
[2]: print(df.info())
print(df.shape)
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 126226 entries, 0 to 126225
Data columns (total 30 columns):
#   Column          Non-Null Count  Dtype

```

```

---  -----  -----  -----
0   r_def      126226 non-null int64
1   cve_ent    126226 non-null int64
2   con        126226 non-null int64
3   upm        126226 non-null int64
4   n_pro_viv  126226 non-null int64
5   v_sel      126226 non-null int64
6   n_hog      126226 non-null int64
7   h_mud      126226 non-null int64
8   n_ren      126226 non-null int64
9   c_res      126226 non-null int64
10  sex        126226 non-null int64
11  eda        126226 non-null int64
12  n_hij      126226 non-null object
13  e_con      126226 non-null int64
14  ur         126226 non-null int64
15  zona       126226 non-null int64
16  pos_ocu    126226 non-null int64
17  seg_soc    126226 non-null int64
18  rama_est1  126226 non-null int64
19  ambito2    126226 non-null int64
20  tue1       126226 non-null int64
21  pre_asa    126226 non-null int64
22  niv_ins    126226 non-null int64
23  anios_esc  126226 non-null int64
24  hrsocup    126226 non-null int64
25  ingocup    126226 non-null int64
26  imssissste 126226 non-null int64
27  emp_ppal   126226 non-null int64
28  p6c        126226 non-null object
29  p6d        126226 non-null int64
dtypes: int64(28), object(2)
memory usage: 28.9+ MB
None
(126226, 30)

```

Tenemos **126,226** observaciones y **29** columnas. Dos cosas saltan de inmediato en el `info()`:

- Casi todo se leyó como `int64`, incluidas variables que en realidad son cualitativas (`sex`, `pos_ocu`, `rama_est1`). Esto confirma lo que se anticipó en la metodología: **el tipo de dato que reporta pandas no es el tipo estadístico de la variable**, y confiar en el `info()` para decidir qué es cuantitativo sería un error.
- Dos columnas (`n_hij` y `p6c`) se leyeron como `object`, o sea texto, a pesar de que deberían ser numéricas. Cuando eso pasa es señal de que la columna tiene algún valor no numérico escondido. Se investiga en la sección de limpieza.

Antes de seguir, hay que separar las columnas que son **identificadores** de las que son **variables del fenómeno**. Las primeras diez columnas (`r_def`, `cve_ent`, `con`, `upm`, `n_pro_viv`, `v_sel`, `n_hog`, `h_mud`, `n_ren`, `c_res`) son la llave primaria y las banderas de control que se usaron para unir las tablas y aplicar los filtros de calidad. Ya cumplieron su función: meterlas al modelo no tendría sentido, porque el número de renglón dentro de un hogar no explica nada sobre el ingreso.

```

[3]: ids = ['r_def', 'c_res', 'cve_ent', 'con', 'upm', 'n_pro_viv',
          'v_sel', 'n_hog', 'h_mud', 'n_ren']
df = df.drop(columns=ids)

print(df.shape)
print(list(df.columns))

```

(126226, 20)

['sex', 'eda', 'n_hij', 'e_con', 'ur', 'zona', 'pos_ocu', 'seg_soc',
 'rama_est1', 'ambito2', 'tue1', 'pre_asa', 'niv_ins', 'anios_esc', 'hrsocup',
 'ingocup', 'imsssisste', 'emp_ppal', 'p6c', 'p6d']

Tabla 2. Diccionario de datos

Variable	Tipo	Descripción y códigos
ingocup	Cuantitativa	Variable de respuesta. Ingreso mensual por ocupación, en pesos
sex	Cualitativa	Sexo: 1 = hombre, 2 = mujer
eda	Cuantitativa	Edad en años cumplidos (12 a 98)
n_hij	Cuantitativa	Número de hijos nacidos vivos
e_con	Cualitativa	Estado conyugal: 1 = unión libre, 2 = separado, 3 = divorciado, 4 = viudo, 5 = casado, 6 = soltero, 9 = no especificado
ur	Cualitativa	Ámbito de residencia: 1 = urbano, 2 = rural
zona	Cualitativa	Zona salarial: 1 = Zona Libre de la Frontera Norte, 2 = resto del país
pos_ocu	Cualitativa	Posición en la ocupación: 1 = subordinado remunerado, 2 = empleador, 3 = trabajador por cuenta propia
seg_soc	Cualitativa	Condición de acceso a seguridad social: 1 = con acceso, 2 = sin acceso, 3 = no especificado
rama_est1	Cualitativa	Sector de actividad: 1 = primario, 2 = secundario, 3 = terciario, 4 = no especificado
ambito2	Cualitativa	Tipo de unidad económica donde labora (8 categorías)
tue1	Cualitativa	Tipo de unidad económica agrupado: 1 = empresas y negocios, 2 = instituciones, 3 = sector informal y hogares, 4 = no especificado
niv_ins	Cualitativa ordinal	Nivel de instrucción: 1 = primaria incompleta, 2 = primaria completa, 3 = secundaria completa, 4 = medio superior y superior, 5 = no especificado
anios_esc	Cuantitativa	Años de escolaridad acumulados (0 a 24; 99 = no especificado)
hrsocup	Cuantitativa	Horas trabajadas a la semana en la ocupación principal
imsssisste	Cualitativa	Institución de servicio médico por el empleo: 1 = IMSS, 2 = ISSSTE, 3 = otra, 4 = sin acceso, 5 = no especificado
emp_ppal	Cualitativa	Condición de informalidad del empleo principal: 1 = informal, 2 = formal
p6c	Cualitativa	Si la cantidad obtenida al mes es mayor, igual o menor al salario mínimo
p6d	Cualitativa	Acceso a atención médica por parte del trabajo (7 categorías)

Variable	Tipo	Descripción y códigos
pre_asa	Cualitativa	Prestaciones, 0 - No aplica, 1 - Con Prestaciones, 2 - Sin Prestaciones, 3 - No especificado

#Preparación y limpieza de datos

##Variables Cualitativas En el `info()` vimos que `n_hij` y `p6c` quedaron como `object`. Eso solo pasa cuando la columna tiene algún valor que no es número, así que lo primero es ver qué valores contienen.

- `.unique()` regresa la lista de valores distintos que aparecen en la columna
- Se imprime también `value_counts()` para ver la frecuencia de cada uno

```
[4]: print("Valores unicos de p6c:", df['p6c'].unique())
print()
print("Valores unicos de n_hij:", sorted(df['n_hij'].astype(str).unique()))
print()
print("n_hij - frecuencia de cada valor:")
print(df['n_hij'].value_counts(dropna=False).head(10))
```

Valores unicos de p6c: [' ']

Valores unicos de n_hij: [' ', '0', '1', '10', '11', '12', '13', '14', '15', '16', '2', '3', '4', '5', '6', '7', '8', '9', '99']

n_hij - frecuencia de cada valor:

```
n_hij
72838
0    13891
2    13431
3    10315
1     8912
4     3869
5     1544
6      689
7      326
8       192
```

Name: count, dtype: int64

La base trae dos columnas con muchos datos faltantes, `p6c` y `n_hij`, pero en ambas el problema no es aleatorio sino que responde al diseño del cuestionario.

p6c: - Tiene un solo valor en toda la base: espacio en blanco. Las 126,226 observaciones están vacías. - Es una pregunta condicionada que solo se le hace a un subconjunto muy chico, y nadie del corte final la contestó. - Al revisar qué pregunta realmente, resultó ser si el ingreso mensual es mayor, igual o menor al salario mínimo, es decir, es el propio ingreso discretizado en categorías. - Aunque hubiera venido llena, usarla sería fuga de datos: la R^2 saldría alta sin que el modelo aprenda nada real, solo estaría leyendo el ingreso disfrazado. - Por estar vacía y ser fuga de datos, se elimina.

n_hij: - Tiene 72,838 valores en blanco, el mismo número de hombres en la muestra. - Se confirma con una tabla cruzada: `pd.to_numeric(..., errors='coerce')` convierte a número o a NaN, y `pd.crosstab(...)` cruza sexo contra si el valor es nulo. - El resultado muestra que la pregunta de hijos nacidos vivos solo se hace a mujeres, así que el hueco es estructural, no aleatorio, y no tiene caso imputarlo. - Al convertir a numérica también aparece un 99, que es código de no respuesta y no una cantidad real de hijos. - Como de todos modos la columna se elimina por el hueco estructural, no hace falta corregir el 99 aparte.

```
[5]: df['n_hij'] = pd.to_numeric(df['n_hij'], errors='coerce')

print(pd.crosstab(df['sex'], df['n_hij'].isna(),
```

```
rownames=['sex'], colnames=['n_hij es nulo']))
```

```
n_hij es nulo  False  True
sex
1              0  72838
2          53388      0
```

```
[6]: df = df.drop(columns=['p6c', 'n_hij'])
```

##Outliers El INEGI no deja las no respuestas en blanco: las **codifica con números**, esto lo podemos ver en el diccionario de datos. Se puede ver que por ejemplo dice que rama de estudio, cuando es 4, significa que no esta especificado. Para pandas esos son números válidos, así que si no se tratan, el modelo va a creer literalmente que hay gente con **99 años de escolaridad**, y esos valores van a jalar los coeficientes hacia donde no deben.

- Se cuenta cuántos casos hay de cada código de no respuesta, para dimensionar el problema antes de decidir qué hacer
- Los códigos se tomaron del diccionario oficial de cada variable

```
[7]: print("anios_esc == 99 (no especificado):", (df['anios_esc'] == 99).sum())
print("e_con == 9 (no especificado):      ", (df['e_con'] == 9).sum())
print("rama_est1 == 4 (no especificado): ", (df['rama_est1'] == 4).sum())
print("tue1 == 4 (no especificado):      ", (df['tue1'] == 4).sum())
print("niv_ins == 5 (no especificado):    ", (df['niv_ins'] == 5).sum())
print("pre_asa == 0 (no aplica):         ", (df['pre_asa'] == 0).sum())
print("pre_asa == 3 (no especificado):    ", (df['pre_asa'] == 3).sum())
```

```
anios_esc == 99 (no especificado): 332
e_con == 9 (no especificado):      11
rama_est1 == 4 (no especificado): 506
tue1 == 4 (no especificado):      480
niv_ins == 5 (no especificado):    258
pre_asa == 0 (no aplica):         32028
pre_asa == 3 (no especificado):    458
```

Los casos de `anios_esc`, `e_con`, `rama_est1` y `tue1` son pocos frente al tamaño de la muestra (332, 11, 506 y 480 respectivamente), así que ahí eliminar es mejor que imputar. El caso de `pre_asa` es distinto: son 32,376 observaciones, más del 25% de la muestra en este punto, y esa proporción tan alta no es casualidad. `pre_asa` (prestaciones laborales) es una pregunta que no aplica a todo tipo de trabajador, típicamente no se le pregunta a quienes trabajan por cuenta propia o son empleadores, solo a subordinados.

Teniendo esto en cuenta vamos a eliminar la columna entera mejor, no nos importa si tienen prestaciones o no, y muchas personas no aplica por trabajar por su cuenta o ser contratistas.

```
[8]: df['anios_esc'] = df['anios_esc'].replace(99, np.nan)
df = df.dropna(subset=['anios_esc'])
df = df[df['e_con'] != 9]
df = df[df['rama_est1'] != 4]
df = df[df['tue1'] != 4]
df = df[df['niv_ins'] != 5]
df = df.drop(columns='pre_asa')
```

```
df.shape
```

```
[8]: (125351, 17)
```

Con todos estos cambios ahora tenemos 17 columnas y 125,351 observaciones, fue lo correcto quitar prestaciones aunque aun no llegamos a selección de variables, porque quitar 25% de la muestra no va a ayudar a nuestro modelo

Ahora vamos a aplicar el método de Tukey a cada variable cuantitativa:

- Se calculan el primer cuartil ($Q1$) y el tercer cuartil ($Q3$), y con ellos el rango intercuartílico ($IQR = Q3 - Q1$)
- Se definen un límite inferior ($Q1 - 1.5 \cdot IQR$) y uno superior ($Q3 + 1.5 \cdot IQR$)
- Se cuentan las observaciones fuera de ese rango, junto con el porcentaje que representan

Este metodo de tukey se va a hacer solo para las variables cuantitativas que sean continuas que son: ingocup, eda, anios_esc y hrsocup

```
[9]: #Celda de codigo hecha con claude
cuant = ['ingocup', 'eda', 'anios_esc', 'hrsocup']
for col in cuant:
    q1 = df[col].quantile(0.25)
    q3 = df[col].quantile(0.75)
    iqr = q3 - q1
    li = q1 - 1.5*iqr
    ls = q3 + 1.5*iqr
    out = df[(df[col] < li) | (df[col] > ls)]
    print(f"{col:10s}: limite inferior={li:9.1f}, limite superior={ls:9.1f}, "
          f"fuera de rango={len(out):6d} ({len(out)/len(df)*100:.2f}%)")
```

```
ingocup   : limite inferior= -3870.0, limite superior= 23650.0, fuera de
rango=    7738 (6.17%)
eda       : limite inferior=   -4.0, limite superior=    84.0, fuera de
rango=     139 (0.11%)
anios_esc : limite inferior=    3.0, limite superior=   19.0, fuera de
rango=   5090 (4.06%)
hrsocup   : limite inferior=    3.0, limite superior=   75.0, fuera de
rango=   8507 (6.79%)
```

Las cuatro variables son continuas, y en las cuatro pasa lo mismo: Tukey marca como atípico todo lo que se aleja de la mayoría, pero alejarse de la mayoría no significa estar mal capturado.

Existen personas de altos ingresos, personas mayores que siguen ocupadas, gente con posgrado, y jornadas extremas en empleos informales. Todos estos datos sin contar hrsocup, son datos que suben gradualmente, no suben de 1000 a un millon por ejemplo entonces ahí va el recap:

No eliminadas: * De ingocup no se va a eliminar nada porque hay variación de ingresos * De edad tampoco porque hay personas que pueden tener mucha edad y ya estar trabajando * De años en escuela tampoco vamos eliminar variables porque personas pueden tener muchos o pocos años en la escuela

Eliminadas: * Hrsocup, vamos a eliminar las personas que tienen más de 119 horas a la semana trabajadas(17 horas por día), porque por más que haya personas con mucha necesidad y trabajadoras, no hay manera que todas las semanas trabajen 17 horas al día, lo cual son 17 filas

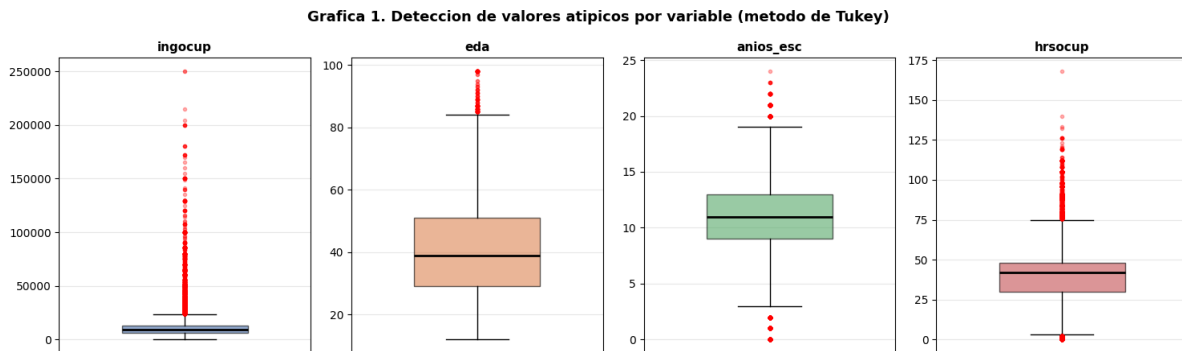
```
[10]: #Celda de codigo hecha con claude
colores = ["#4C72B0", "#DD8452", "#55A868", "#C44E52"]

fig, axes = plt.subplots(1, len(cuant), figsize=(15, 4.5))
for ax, col, color in zip(axes, cuant, colores):
    ax.boxplot(df[col], patch_artist=True, widths=0.5,
               boxprops=dict(facecolor=color, alpha=0.6, edgecolor="black"),
               medianprops=dict(color="black", linewidth=2),
               flierprops=dict(marker="o", markerfacecolor="red",
                               markeredgecolor="red", markersize=3, alpha=0.3))
    ax.set_title(col, fontsize=11, fontweight="bold")
    ax.set_xticks([])
    ax.grid(axis="y", alpha=0.3)

fig.suptitle("Grafica 1. Deteccion de valores atipicos por variable (metodo de Tukey)",
             fontsize=13, fontweight="bold")
```



```
plt.tight_layout()
plt.show()
```



```
[11]: df.shape
print((df['hrsocup'] >= 119).sum())
df = df[df['hrsocup'] <= 119]
df.shape
```

25

```
[11]: (125336, 17)
```

Aquí al quitar los 25 que tienen más de 119 horas ahora tenemos 125,336 en la muestra de datos, no es mucho pero comoquiera fue lo correcto quitar esos outliers

2.1.3 Gestión de ceros

Vamos a revisar que variables continuas tienen 0s, ya que en algunas no importa si tienen 0, y en otras si puede ser algo raro

- En **anios_esc** un 0 sí es un valor que puede pasar: puede haber personas ocupadas, con ingreso, que nunca fueron a la escuela.
- En **hrsocup** un 0 es distinto y sí es un problema: toda la muestra ya son personas que trabajan, así que alguien con 0 horas trabajadas en la semana pero con ingreso reportado es un caso contradictorio para el modelo, no una jornada real. Por eso estas filas se eliminan.

```
[12]: for col in quant:
    ceros = (df[col] == 0).sum()
    print(f"{col:10s}: {ceros} ceros ({ceros/len(df)*100:.2f}%)")
```

```
ingocup   : 0 ceros (0.00%)
eda       : 0 ceros (0.00%)
anios_esc : 2867 ceros (2.29%)
hrsocup   : 4941 ceros (3.94%)
```

```
[13]: df = df[df['hrsocup'] != 0]
df.shape
```

```
[13]: (120395, 17)
```

Ahora la muestra nos queda como 120,395 observaciones con 17 variables, esto si fue un poco más que los outliers, pero comoquiera nos ayuda quitar mas o menos un 4 por ciento de los datos

Bajo de 125,336 a 120,395

2.1.4 Colinealidad

Antes de estudiar la colinealidad entre las variables explicativas, dividimos el conjunto de datos en entrenamiento (80%) y prueba (20%), pues a partir de este punto **todo el análisis de relación entre variables y la selección de características se hará únicamente con el subconjunto de entrenamiento**, dejando el subconjunto de prueba completamente al margen hasta la evaluación final del modelo. Esta es la medida principal que se tomó en este reporte para evitar fuga de datos.

- `train_test_split` (de `sklearn.model_selection`) divide el data frame en dos subconjuntos de forma aleatoria
- `train_size=0.8` indica que el 80% de las observaciones va a `train` y el 20% restante a `test`
- `random_state=67` fija la semilla del generador aleatorio, para que la división sea siempre la misma cada vez que se corra el código (reproducibilidad)
- `train.shape` y `test.shape` confirman cuántas observaciones y columnas quedaron en cada subconjunto

```
[14]: from sklearn.model_selection import train_test_split

train, test = train_test_split(df, train_size=0.8, random_state=67)
print(train.shape)
print(test.shape)
```

```
(96316, 17)
```

```
(24079, 17)
```

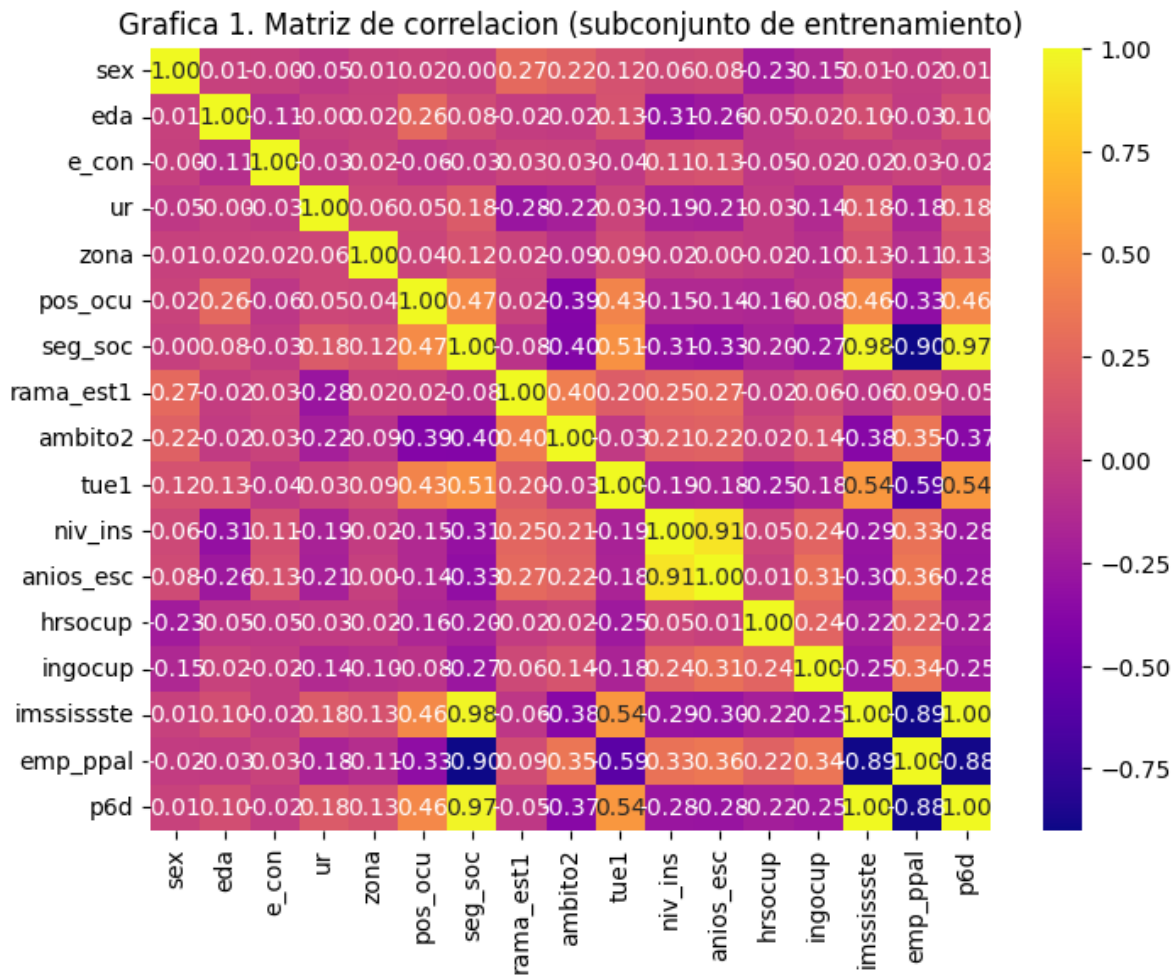
Quedamos con 96316 observaciones de entrenamiento y 24079 de prueba. Ahora sí, generamos la matriz de correlación de Pearson entre todas las variables (incluyendo `ingreso mensual`, para tener una primera idea de qué variables podrían ser más relevantes) usando únicamente `train`, y la visualizamos con un *heatmap* de la librería `seaborn`

2.1.5 Colinealidad entre variables de formalidad y acceso a salud

1. `imsssisste` y `p6d` tienen correlación de 1.00. Según el diccionario, `imsssisste` es la institución de servicio médico y `p6d` es el acceso a atención médica por parte del trabajo, entonces podemos concluir que son exactamente lo mismo. Vamos a eliminar `p6d`
2. `seg_soc` y `imsssisste` tienen correlación de 0.98. `seg_soc` es la variable precodificada oficial del INEGI para condición de acceso a seguridad social, y básicamente resume lo mismo que `imsssisste` pero de forma más simple y ya limpia. Por razones que prefiero la compleja, se elimina `seg_soc`
3. `seg_soc` y `p6d` tienen correlación de 0.97, mismo caso: ambas describen si la persona tiene acceso a servicios médicos por su trabajo. Como ya se decidió eliminar `p6d` en el punto 1, este par queda resuelto también.
4. `imsssisste` y `emp_ppal` tienen correlación de 0.89. `emp_ppal` es la condición de informalidad del empleo principal (informal/formal), un concepto relacionado pero no idéntico a tener acceso al IMSS o ISSSTE: se puede ser formal y no tener seguridad social por el empleo en algunos casos, así que esta variable sí aporta algo distinto y se conserva.
5. `emp_ppal` y `p6d` tienen correlación de 0.88, ya se eliminó `p6d` entonces no hay problema
6. `niv_ins` tiene correlación de 0.91 con `anios_esc`. Ambas miden escolaridad, pero `niv_ins` la agrupa en solo 4 categorías ordinales mientras que `anios_esc` la mide en años exactos (una escala más fina y con más información). Entonces se elimina `niv_ins`

```
[15]: #Codigo hecho con claude
import matplotlib.pyplot as plt
from seaborn import heatmap

corrTrain = train.corr()
plt.figure(figsize=(8,6))
heatmap(corrTrain, annot=True, fmt=".2f", cmap="plasma")
plt.title("Grafica 1. Matriz de correlacion (subconjunto de entrenamiento)")
plt.show()
```



Con todos estos cambios el train y test en observaciones se queda igual pero el numero de columnas cambia a 14

```
[16]: train = train.drop(columns=['p6d', 'seg_soc', 'niv_ins'])
test = test.drop(columns=['p6d', 'seg_soc', 'niv_ins'])
print(train.shape)
print(test.shape)
```

(96316, 14)

(24079, 14)

2.1.6 Código de no respuesta pendiente en imsssisste

Al revisar de nuevo el diccionario de datos, `imsssisste` también tiene un código de no especificado (5) que no se filtró en la sección de Outliers, porque en ese momento no se había detectado. Como esta variable sí se conserva después del análisis de colinealidad, es necesario limpiarla ahora, antes de convertir las variables cualitativas a dummies.

```
[17]: print("imsssisste == 5 (no especificado):", (train['imsssisste'] == 5).sum())

train = train[train['imsssisste'] != 5]
test = test[test['imsssisste'] != 5]

print(train.shape)
print(test.shape)
```

```
imssissste == 5 (no especificado): 182
(96134, 14)
(24033, 14)
```

Quitando esto, ahora tenemos 14 variables en train y test, y 96134 observaciones en train y 24033 en test

2.1.7 Codificación de variables cualitativas (dummies)

Los modelos de regresión necesitan que todas las variables explicativas sean numéricas con un sentido matemático real, y varias de las que quedan en el dataframe son cualitativas codificadas como números (por ejemplo, `rama_est1` con 1 = primario, 2 = secundario, 3 = terciario), donde el número es solo una etiqueta y no una cantidad. Si se dejan así, el modelo interpretaría, por ejemplo, que el sector terciario “vale el triple” que el primario, lo cual no tiene ningún sentido.

Las variables cualitativas del dataframe se dividen en dos grupos:

- **Binarias** (`sex`, `ur`, `zona`, `emp_ppal`): solo tienen dos categorías, así que basta con recodificarlas a 0 y 1, sin necesidad de crear columnas adicionales.
- **De más de dos categorías** (`e_con`, `rama_est1`, `pos_ocu`, `ambito2`, `imssissste`, `tue1`): estas sí necesitan convertirse en varias columnas dummy, una por categoría, dejando una fuera como categoría de referencia (`drop_first=True`), para evitar la trampa de las variables dummy (colinealidad perfecta si se incluyen todas).

```
[18]: train['sex'] = train['sex'].replace({1: 0, 2: 1})
test['sex'] = test['sex'].replace({1: 0, 2: 1})

train['ur'] = train['ur'].replace({1: 0, 2: 1})
test['ur'] = test['ur'].replace({1: 0, 2: 1})

train['zona'] = train['zona'].replace({1: 0, 2: 1})
test['zona'] = test['zona'].replace({1: 0, 2: 1})

train['emp_ppal'] = train['emp_ppal'].replace({1: 0, 2: 1})
test['emp_ppal'] = test['emp_ppal'].replace({1: 0, 2: 1})

print(train[['sex', 'ur', 'zona', 'emp_ppal']].head())
```

	sex	ur	zona	emp_ppal
66066	1	1	1	0
24624	0	1	1	1
24297	1	1	1	1
123214	0	1	1	0
112865	0	0	1	0

Para las variables binarias, 0 representa la primera categoría del diccionario y 1 la segunda: `sex` (0 = hombre, 1 = mujer), `ur` (0 = urbano, 1 = rural), `zona` (0 = Zona Libre de la Frontera Norte, 1 = resto del país) y `emp_ppal` (0 = informal, 1 = formal). Esto define directamente cómo se van a interpretar los coeficientes de esas variables en el modelo.

```
[19]: cat_multi = ['e_con', 'rama_est1', 'pos_ocu', 'ambito2', 'imssissste', 'tue1']

train = pd.get_dummies(train, columns=cat_multi, drop_first=True)
test = pd.get_dummies(test, columns=cat_multi, drop_first=True)

print(train.shape)
print(test.shape)
print(list(train.columns))
```

```
(96134, 29)
(24033, 29)
['sex', 'eda', 'ur', 'zona', 'anios_esc', 'hrsocup', 'ingocup', 'emp_ppal',
'e_con_2', 'e_con_3', 'e_con_4', 'e_con_5', 'e_con_6', 'rama_est1_2',
```

```
'rama_est1_3', 'pos_ocu_2', 'pos_ocu_3', 'ambito2_2', 'ambito2_3', 'ambito2_4',
'ambito2_5', 'ambito2_6', 'ambito2_7', 'ambito2_8', 'imssissste_2',
'imssissste_3', 'imssissste_4', 'tue1_2', 'tue1_3']
```

Con `drop_first=True`, la primera categoría de cada variable (según el orden de sus códigos) queda como referencia implícita y no genera columna propia; el resto de las categorías se interpretan como el efecto de pertenecer a ese grupo en comparación con la categoría de referencia. Por ejemplo, para `rama_est1` la categoría 1 (sector primario) es la referencia, así que los coeficientes de `rama_est1_2` y `rama_est1_3` se leerán como el efecto de estar en el sector secundario o terciario respecto al primario.

Un detalle importante de reproducibilidad: `train` y `test` se codificaron por separado. Si alguna categoría existiera en uno de los dos subconjuntos pero no en el otro, terminarían con columnas dummy distintas y el modelo fallaría al evaluar sobre `test`. Vale la pena confirmar con `set(train.columns) - set(test.columns)` que las columnas resultantes son exactamente las mismas en ambos.

```
[20]: print("Columnas en train pero no en test:", set(train.columns) - set(test.columns))
      print("Columnas en test pero no en train:", set(test.columns) - set(train.columns))
```

```
Columnas en train pero no en test: set()
Columnas en test pero no en train: set()
```

Este pequeño error se me paso por eso lo hice despues de train test.

Un detalle técnico antes de entrenar el modelo: `pd.get_dummies()` genera las columnas nuevas con tipo `bool` (`True/False`) en vez de números. Al mezclarse con las columnas `int64` del resto del dataframe, `statsmodels` interpreta todo el conjunto como tipo `object`, lo cual no puede procesar. Para evitarlo, se convierten explícitamente las columnas booleanas a enteros (0 y 1) antes de entrenar cualquier modelo.

```
[22]: for col in train.columns:
      if train[col].dtype == bool:
          train[col] = train[col].astype(int)

      for col in test.columns:
          if test[col].dtype == bool:
              test[col] = test[col].astype(int)

      print(train.dtypes.value_counts())
```

```
int64      28
float64     1
Name: count, dtype: int64

#Selección de características
```

Empezamos generando un modelo de referencia con las 13 variables explicativas disponibles (`sex`, `eda`, `ur`, `zona`, `anios_esc`, `hrsocup`, `emp_ppal`, `e_con`, `rama_est1`, `pos_ocu`, `ambito2`, `imssissste`, `tue1`), entrenado únicamente con el subconjunto de entrenamiento, para tener un punto de comparación contra el que evaluar si la selección de características logra simplificar el modelo sin perder desempeño.

- `X_train = train.drop(columns=["ingocup"])` separa las variables explicativas, quitando `ingocup` (la variable de respuesta) del subconjunto de entrenamiento
- `Y_train = train["ingocup"]` guarda `ingocup` por separado, como la variable que se quiere predecir
- `sm.add_constant(X_train)` agrega una columna de puros 1's a `X_train`, necesaria para que el modelo estime el intercepto (β_0)
- `sm.OLS(Y_train, ...)` define el modelo de regresión lineal por mínimos cuadrados ordinarios (*Ordinary Least Squares*), usando `Y_train` como respuesta
- `.fit()` entrena el modelo, estimando los coeficientes que minimizan la suma de errores al cuadrado
- `.summary()` imprime una tabla con los coeficientes, sus p-values, la R^2 , la R^2 ajustada y el estadístico F del modelo completo

```
[23]: import statsmodels.api as sm
```

```
X_train = train.drop(columns=["ingocup"])
```

```
Y_train = train["ingocup"]
```

```
modelFull = sm.OLS(Y_train, sm.add_constant(X_train)).fit()
```

```
print(modelFull.summary())
```

OLS Regression Results

Dep. Variable:	ingocup	R-squared:	0.265			
Model:	OLS	Adj. R-squared:	0.264			
Method:	Least Squares	F-statistic:	1235.			
Date:	Wed, 02 Sep 2026	Prob (F-statistic):	0.00			
Time:	21:09:03	Log-Likelihood:	-9.9567e+05			
No. Observations:	96134	AIC:	1.991e+06			
Df Residuals:	96105	BIC:	1.992e+06			
Df Model:	28					
Covariance Type:	nonrobust					
=====						
	coef	std err	t	P> t	[0.025	0.975]

const	-662.2698	249.711	-2.652	0.008	-1151.700	-172.839
sex	-2159.4454	55.538	-38.882	0.000	-2268.300	-2050.591
eda	26.7819	2.196	12.196	0.000	22.478	31.086
ur	-859.7890	55.024	-15.626	0.000	-967.636	-751.942
zona	-2297.7651	95.148	-24.149	0.000	-2484.254	-2111.277
anios_esc	508.9640	7.267	70.042	0.000	494.722	523.206
hrsocup	88.2245	1.624	54.336	0.000	85.042	91.407
emp_ppal	4261.7467	161.128	26.449	0.000	3945.937	4577.557
e_con_2	-877.2539	110.782	-7.919	0.000	-1094.386	-660.122
e_con_3	-323.7023	158.840	-2.038	0.042	-635.027	-12.377
e_con_4	-1045.5853	156.724	-6.672	0.000	-1352.763	-738.408
e_con_5	-20.5023	68.502	-0.299	0.765	-154.765	113.761
e_con_6	-1512.6874	71.011	-21.302	0.000	-1651.869	-1373.506
rama_est1_2	1906.9664	174.874	10.905	0.000	1564.215	2249.717
rama_est1_3	1171.1070	170.925	6.852	0.000	836.096	1506.118
pos_ocu_2	5680.3509	146.698	38.722	0.000	5392.825	5967.876
pos_ocu_3	407.2545	93.304	4.365	0.000	224.379	590.130
ambito2_2	-140.4342	166.007	-0.846	0.398	-465.806	184.937
ambito2_3	-519.0706	148.853	-3.487	0.000	-810.820	-227.321
ambito2_4	684.7572	140.618	4.870	0.000	409.148	960.367
ambito2_5	1333.9371	150.113	8.886	0.000	1039.717	1628.158
ambito2_6	1948.9114	149.672	13.021	0.000	1655.556	2242.267
ambito2_7	356.4868	197.872	1.802	0.072	-31.340	744.313
ambito2_8	181.9315	201.268	0.904	0.366	-212.551	576.414
imssissste_2	2820.9378	149.548	18.863	0.000	2527.825	3114.051
imssissste_3	246.7995	365.669	0.675	0.500	-469.908	963.507
imssissste_4	2455.8895	167.347	14.675	0.000	2127.891	2783.888
tue1_2	696.6707	126.566	5.504	0.000	448.603	944.738
tue1_3	-161.3736	111.673	-1.445	0.148	-380.252	57.505
=====						
Omnibus:	114474.582	Durbin-Watson:	1.999			
Prob(Omnibus):	0.000	Jarque-Bera (JB):	32842839.696			
Skew:	6.046	Prob(JB):	0.00			
Kurtosis:	92.739	Cond. No.	1.00e+03			

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 1e+03. This might indicate that there are strong multicollinearity or other numerical problems.

En el modelo completo de arriba, con las 13 variables (28 dummies), la R2 ajustada salió en 0.2643. Hay variables que claramente dominan el modelo: pos_ocu_2, emp_ppal, zona, sex y anios_esc, todas con p-values practicamente en cero. Pero tambien hay otras que no salieron significativas por si solas, como e_con_5, ambito2_2, ambito2_7, ambito2_8, inssissste_3 y tue1_3, con p-values arriba de 0.05.

Como ya vimos que el modelo completo tiene dummies que no son significativas solas, el siguiente paso es hacer seleccion de caracteristicas. En lugar de decidir a mano cuales quitar, vamos a ordenar las variables segun que tanto se relacionan con ingocup, y usar ese orden para ir probando modelos mas chicos.

La razon de ordenar primero es que con 13 variables candidatas, probar todas las combinaciones posibles seria 2 elevado a la 13, osea 8192 modelos. Por eso mejor usamos la seleccion hacia adelante rapida, que fija el orden desde el inicio segun la correlacion y no lo vuelve a checar. Esto la hace mas rapida pero no garantiza encontrar el mejor subconjunto posible, por eso mas adelante tambien hacemos una eliminacion hacia atras sobre el resultado, para corregir esa debilidad.

Aqui hay una diferencia con el caso de referencia: varias variables explicativas son cualitativas con mas de dos categorias, como e_con, rama_est1, pos_ocu, ambito2, inssissste y tue1, y cada una esta representada por varias columnas dummy. Si calculamos la correlacion de cada dummy por separado y las ordenamos sueltas, podria pasar que se incluya una categoria de una variable pero no otra, por ejemplo meter rama_est1_2 pero no rama_est1_3. Eso significaria tratar la categoria que se quedo fuera como si fuera igual a la categoria de referencia, lo cual no tiene mucho sentido. Por eso el ranking y la seleccion se hacen por bloques, es decir, todas las dummies de una misma variable entran juntas o no entra ninguna.

Para hacerlo, primero agrupamos las columnas dummy segun la variable cualitativa de la que vienen. Luego calculamos la correlacion de Pearson en valor absoluto entre cada dummy y ingocup, usando solo train, y nos quedamos con la correlacion mas alta dentro de cada bloque como su valor representativo. Con eso ordenamos los bloques de mayor a menor correlacion, y vamos entrenando un modelo por cada cantidad de bloques agregados, guardando la R2 ajustada de cada uno. Al final graficamos esos resultados para ver en que punto ya no vale la pena seguir agregando mas bloques.

```
[24]: #Celda de codigo hecha con claude
bloques = {}
for col in X_train.columns:
    if col.startswith('e_con_'):         base = 'e_con'
    elif col.startswith('rama_est1_'):   base = 'rama_est1'
    elif col.startswith('pos_ocu_'):     base = 'pos_ocu'
    elif col.startswith('ambito2_'):     base = 'ambito2'
    elif col.startswith('imssissste_'):  base = 'imssissste'
    elif col.startswith('tue1_'):        base = 'tue1'
    else:                                 base = col
    bloques.setdefault(base, []).append(col)

for base, cols in bloques.items():
    print(f"{base:12s} -> {cols}")
```

```
sex          -> ['sex']
eda          -> ['eda']
ur           -> ['ur']
zona         -> ['zona']
anios_esc    -> ['anios_esc']
hrsocup      -> ['hrsocup']
emp_ppal     -> ['emp_ppal']
e_con        -> ['e_con_2', 'e_con_3', 'e_con_4', 'e_con_5', 'e_con_6']
rama_est1    -> ['rama_est1_2', 'rama_est1_3']
```



```
pos_ocu      -> ['pos_ocu_2', 'pos_ocu_3']
ambito2      -> ['ambito2_2', 'ambito2_3', 'ambito2_4', 'ambito2_5',
'ambito2_6', 'ambito2_7', 'ambito2_8']
imssissste   -> ['imssissste_2', 'imssissste_3', 'imssissste_4']
tue1         -> ['tue1_2', 'tue1_3']
```

```
[25]: #Celda de código hecha con claude
corr_target = train.corr()['ingocup'].drop('ingocup')

ranking = [(base, corr_target[cols].abs().max()) for base, cols in bloques.items()]
ranking.sort(key=lambda t: -t[1])

for base, mc in ranking:
    print(f"{base:12s} max|corr|={mc:.4f}")
```

```
emp_ppal      max|corr|=0.3445
anios_esc     max|corr|=0.3136
imssissste    max|corr|=0.2747
hrsocup       max|corr|=0.2394
tue1          max|corr|=0.2345
ambito2       max|corr|=0.1631
pos_ocu       max|corr|=0.1601
sex           max|corr|=0.1548
ur            max|corr|=0.1390
zona          max|corr|=0.0995
e_con         max|corr|=0.0974
rama_est1     max|corr|=0.0597
eda           max|corr|=0.0149
```

Aquí podemos ver que las mejores variables fueron las siguientes: - emp_ppal con 0.345 y anios_esc con 0.314 - imssissste, hrsocup y tue1 entre 0.235 y 0.275 - ambito2, pos_ocu, sex y ur entre 0.139 y 0.163

Y el resto (zona, e_con, rama_est1, eda) con menos de 0.10.

Ya con esto podemos hacer los modelos empezando desde emp_ppal.

###Selección Rapida hacia adelante

- Radj = np.zeros(len(orden_bloques)) crea un arreglo vacío donde se va a guardar la R^2 ajustada de cada modelo, uno por cada cantidad de bloques
- El ciclo for recorre los bloques en el orden del ranking, acumulando sus dummies
- Se entrena un modelo (sm.OLS(...).fit()) usando solo las dummies acumuladas hasta ese punto
- Radj[i] guarda la R^2 ajustada de ese modelo en la posición correspondiente del arreglo
- Al final, se muestra la R^2 ajustada obtenida con 1 bloque, con 2, con 3... hasta con los 13 completos

```
[26]: orden_bloques = [b for b, _ in ranking]

Radj = np.zeros(len(orden_bloques))
acumuladas = []
for i, base in enumerate(orden_bloques):
    acumuladas += bloques[base]
    modelTemp = sm.OLS(Y_train, sm.add_constant(train[acumuladas])).fit()
    Radj[i] = modelTemp.rsquared_adj

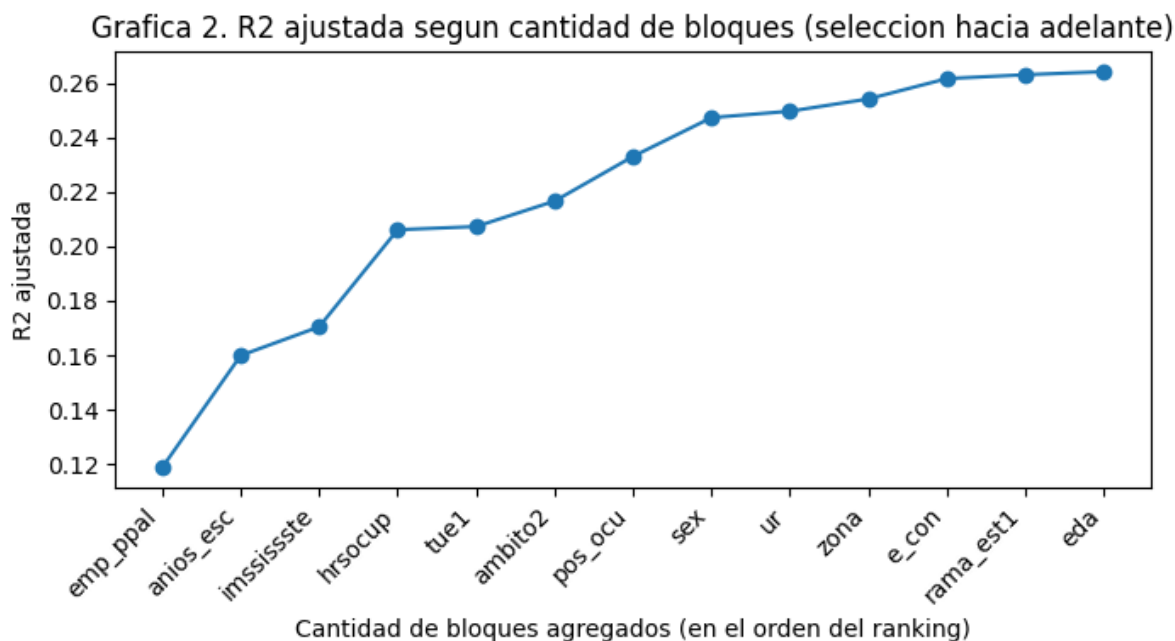
print(Radj)
```

```
[0.11866834 0.15987972 0.17044343 0.20615595 0.20732831 0.21669154
0.23313337 0.24741203 0.24976663 0.2542875 0.26176448 0.26318535
0.26431639]
```

Al graficarla podemos ver que el mejor modelo es el modelo que tiene todas las variables, con un r^2 ajustada de

0.26, despues de esto ya aplicamos selección rápida y nos vamos a ir quitando variables hasta quedar con la mejor r^2 ajustada

```
[27]: #La graficamos
plt.figure(figsize=(7,4))
plt.plot(range(1, len(Radj)+1), Radj, "o-")
plt.xlabel("Cantidad de bloques agregados (en el orden del ranking)")
plt.ylabel("R2 ajustada")
plt.title("Grafica 2. R2 ajustada segun cantidad de bloques (seleccion hacia adelante)")
plt.xticks(range(1, len(orden_bloques)+1), orden_bloques, rotation=45, ha="right")
plt.tight_layout()
plt.show()
```



Eliminación hacia atras - **activos** guarda la lista de bloques que siguen en el modelo - En cada paso, se entrena un modelo por cada bloque activo, quitando ese bloque (todas sus dummies) del conjunto - Se compara la mejor R^2 ajustada obtenida al quitar un bloque contra la R^2 ajustada del modelo con todos los bloques activos - Si quitar algún bloque **mejora** la R^2 ajustada, se elimina ese bloque y se repite el proceso con los bloques restantes - Si ninguna eliminación mejora la R^2 ajustada, el proceso se detiene y el conjunto de bloques activos queda como el modelo final

```
[28]: #Celda de codigo hecha con claude
activos = list(bloques.keys())
modelFull_check = sm.OLS(Y_train, sm.add_constant(X_train)).fit()
r2_actual = modelFull_check.rsquared_adj
paso = 1

while True:
    resultados = {}
    for base in activos:
        cols_restantes = []
        for b in activos:
            if b != base:
                cols_restantes += bloques[b]
        modelTemp = sm.OLS(Y_train, sm.add_constant(train[cols_restantes])).fit()
```

```

resultados[base] = modelTemp.rsquared_adj

print(f"--- Paso {paso} (R2 actual con {len(activos)} bloques = {r2_actual:.4f}) ---")
for base, r2 in resultados.items():
    print(f"    sin {base:12s} -> R2 ajustada = {r2:.4f}")

mejor_base = max(resultados, key=resultados.get)
mejor_r2 = resultados[mejor_base]

if mejor_r2 > r2_actual:
    print(f"    >> Se elimina '{mejor_base}' (mejora de {mejor_r2 - r2_actual:+.4f})")
    activos.remove(mejor_base)
    r2_actual = mejor_r2
    paso += 1
else:
    print("    >> Ninguna eliminacion mejora el modelo. Se detiene.")
    break

```

```

--- Paso 1 (R2 actual con 13 bloques = 0.2643) ---
sin sex          -> R2 ajustada = 0.2528
sin eda          -> R2 ajustada = 0.2632
sin ur           -> R2 ajustada = 0.2625
sin zona         -> R2 ajustada = 0.2599
sin anios_esc    -> R2 ajustada = 0.2268
sin hrsocup      -> R2 ajustada = 0.2417
sin emp_ppal     -> R2 ajustada = 0.2590
sin e_con        -> R2 ajustada = 0.2594
sin rama_est1    -> R2 ajustada = 0.2628
sin pos_ocu      -> R2 ajustada = 0.2512
sin ambito2      -> R2 ajustada = 0.2605
sin imssissste   -> R2 ajustada = 0.2602
sin tue1         -> R2 ajustada = 0.2641
>> Ninguna eliminacion mejora el modelo. Se detiene.

```

Usando esta función para ver si se necesita eliminar variables para atrás, se puede ver que no se tuvo que eliminar ninguna variable, porque el R2 Ajustado de ahorita es de 0.2643 y eliminando cualquier variable, hacia que se hiciera peor el modelo. Entonces ya tenemos las variables que vamos a usar para el modelo final

Ya habíamos deducido cuales variables deberíamos de quitar en la limpieza de datos, pero comoquiera es bueno rechecar usando eliminacion para atrás en seleccion rapida para poder rechecar que si sea correcto nuestra inferencia de cuales variables debemos quitar y cuales debemos de dejar

Tabla 3. Variables seleccionadas para el modelo final

Variable	Descripción	Correlación máxima con <code>ingocup</code> (train)
<code>emp_ppal</code>	Condición de informalidad del empleo principal	0.345
<code>anios_esc</code>	Años de escolaridad acumulados	0.314
<code>imssissste</code>	Institución de servicio médico por el empleo	0.275
<code>hrsocup</code>	Horas trabajadas a la semana	0.239
<code>tue1</code>	Tipo de unidad económica agrupado	0.235
<code>ambito2</code>	Tipo de unidad económica (8 categorías)	0.163
<code>pos_ocu</code>	Posición en la ocupación	0.160
<code>sex</code>	Sexo	0.155
<code>ur</code>	Ámbito de residencia	0.139
<code>zona</code>	Zona salarial	0.099
<code>e_con</code>	Estado conyugal	0.097
<code>rama_est1</code>	Sector de actividad económica	0.060
<code>eda</code>	Edad	0.015

3 Construcción y comparación de modelos

###Regresión Lineal Multiple

El primer modelo es regresion lineal multiple, donde vamos a poner el ingreso en pesos mensual como el Y, y las otras variables como X

```
[30]: import statsmodels.api as sm

modelLineal = sm.OLS(Y_train, sm.add_constant(X_train)).fit()
print(modelLineal.summary())
```

```

=====
                        OLS Regression Results
=====
Dep. Variable:          ingocup      R-squared:                0.265
Model:                  OLS          Adj. R-squared:           0.264
Method:                 Least Squares  F-statistic:             1235.
Date:                   Wed, 02 Sep 2026  Prob (F-statistic):       0.00
Time:                   21:18:43      Log-Likelihood:          -9.9567e+05
No. Observations:      96134         AIC:                    1.991e+06
Df Residuals:          96105         BIC:                    1.992e+06
Df Model:               28
Covariance Type:       nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
const	-662.2698	249.711	-2.652	0.008	-1151.700	-172.839
sex	-2159.4454	55.538	-38.882	0.000	-2268.300	-2050.591
eda	26.7819	2.196	12.196	0.000	22.478	31.086
ur	-859.7890	55.024	-15.626	0.000	-967.636	-751.942
zona	-2297.7651	95.148	-24.149	0.000	-2484.254	-2111.277
anios_esc	508.9640	7.267	70.042	0.000	494.722	523.206
hrsocup	88.2245	1.624	54.336	0.000	85.042	91.407
emp_ppal	4261.7467	161.128	26.449	0.000	3945.937	4577.557
e_con_2	-877.2539	110.782	-7.919	0.000	-1094.386	-660.122
e_con_3	-323.7023	158.840	-2.038	0.042	-635.027	-12.377
e_con_4	-1045.5853	156.724	-6.672	0.000	-1352.763	-738.408
e_con_5	-20.5023	68.502	-0.299	0.765	-154.765	113.761
e_con_6	-1512.6874	71.011	-21.302	0.000	-1651.869	-1373.506
rama_est1_2	1906.9664	174.874	10.905	0.000	1564.215	2249.717
rama_est1_3	1171.1070	170.925	6.852	0.000	836.096	1506.118
pos_ocu_2	5680.3509	146.698	38.722	0.000	5392.825	5967.876
pos_ocu_3	407.2545	93.304	4.365	0.000	224.379	590.130
ambito2_2	-140.4342	166.007	-0.846	0.398	-465.806	184.937
ambito2_3	-519.0706	148.853	-3.487	0.000	-810.820	-227.321
ambito2_4	684.7572	140.618	4.870	0.000	409.148	960.367
ambito2_5	1333.9371	150.113	8.886	0.000	1039.717	1628.158
ambito2_6	1948.9114	149.672	13.021	0.000	1655.556	2242.267
ambito2_7	356.4868	197.872	1.802	0.072	-31.340	744.313
ambito2_8	181.9315	201.268	0.904	0.366	-212.551	576.414
imsssisste_2	2820.9378	149.548	18.863	0.000	2527.825	3114.051
imsssisste_3	246.7995	365.669	0.675	0.500	-469.908	963.507
imsssisste_4	2455.8895	167.347	14.675	0.000	2127.891	2783.888
tue1_2	696.6707	126.566	5.504	0.000	448.603	944.738
tue1_3	-161.3736	111.673	-1.445	0.148	-380.252	57.505

```

=====
Omnibus:                114474.582    Durbin-Watson:           1.999
Prob(Omnibus):           0.000        Jarque-Bera (JB):        32842839.696
Skew:                    6.046        Prob(JB):                0.00
=====

```

Kurtosis: 92.739 Cond. No. 1.00e+03

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 1e+03. This might indicate that there are strong multicollinearity or other numerical problems.

Este es el mismo modelo que ya se entrenó en la sección de selección de características (con las 13 variables que la eliminación hacia atrás confirmó que no se podían simplificar), así que la R^2 ajustada sigue siendo 0.2643.

Antes de interpretar hay que tener claro contra qué se comparan los coeficientes de las dummies. Como se usó `drop_first=True`, la categoría que quedó fuera de cada variable es la referencia:

Tabla 4. Categorías de referencia del modelo

Variable	Categoría de referencia
sex	Hombre (0)
ur	Urbano (0)
zona	Zona Libre de la Frontera Norte (0)
emp_ppal	Informal (0)
e_con	Unión libre
rama_est1	Sector primario
pos_ocu	Subordinado remunerado
ambito2	Categoría 1
imssissste	IMSS
tue1	Empresas y negocios

Con esto, los coeficientes más grandes y significativos ($p < 0.001$) del modelo son: `pos_ocu_2` (ser empleador se asocia con +5,680 pesos respecto a ser subordinado remunerado), `emp_ppal` (tener empleo formal se asocia con +4,262 pesos respecto a informal), `zona` (vivir en el resto del país se asocia con -2,298 pesos respecto a la frontera norte) y `anios_esc` (+509 pesos por cada año adicional de escolaridad). Algunas dummies (`e_con_5`, `ambito2_2`, `ambito2_7`, `ambito2_8`, `imssissste_3`, `tue1_3`) no son significativas de forma individual, pero se conservan porque forman parte de bloques de variables cualitativas cuya inclusión completa ya se justificó en la sección anterior.

3.0.1 Random Forest

El segundo modelo es un bosque aleatorio (*Random Forest*), un modelo no lineal capaz de capturar interacciones entre variables sin que haya que especificarlas a mano (por ejemplo, que el efecto de las horas trabajadas sobre el ingreso sea distinto según la posición en la ocupación).

- `n_estimators=100` entrena 100 árboles de decisión distintos, y el modelo final promedia sus predicciones
- `min_samples_leaf=20` evita que los árboles crezcan tanto que memoricen casos individuales (sobreajuste), exigiendo al menos 20 observaciones por hoja
- `random_state=67` fija la semilla, para que el resultado sea reproducible
- Se entrena únicamente con `X_train/Y_train`, igual que el modelo lineal, para poder comparar ambos de forma justa más adelante

[31]: `from sklearn.ensemble import RandomForestRegressor`

```
modelRF = RandomForestRegressor(
    n_estimators=100,
    min_samples_leaf=20,
    n_jobs=-1,
    random_state=67
)
modelRF.fit(X_train, Y_train)
```

```
print("Bosque aleatorio entrenado con", len(modelRF.estimators_), "arboles.")
```

Bosque aleatorio entrenado con 100 arboles.

El bosque no tiene coeficientes, pero sí una medida de **importancia de cada variable**, que indica cuánto contribuye cada una a reducir el error a lo largo de todos los árboles.

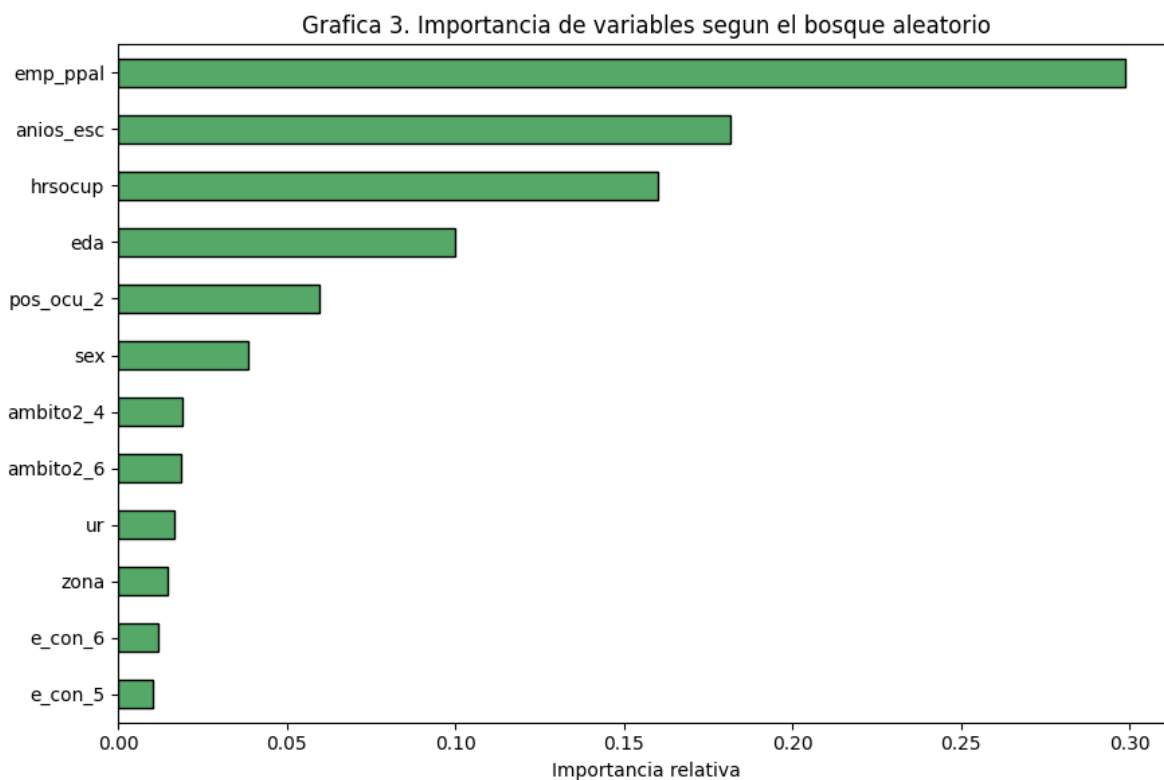
- `feature_importances_` regresa la importancia relativa de cada columna, y todas suman 1
- Se ordenan de mayor a menor con `sort_values` y se grafican las 12 más importantes con un `barh` (barras horizontales)

```
[33]: #Celda de codigo hecha con claude
import pandas as pd

imp = pd.Series(modelRF.feature_importances_, index=X_train.columns)
imp = imp.sort_values(ascending=False)

plt.figure(figsize=(9, 6))
imp.head(12).sort_values().plot(kind='barh', color="#55A868", edgecolor="black")
plt.xlabel("Importancia relativa")
plt.title("Grafica 3. Importancia de variables segun el bosque aleatorio")
plt.tight_layout()
plt.show()

print(imp.head(12).round(4).to_string())
```



```
emp_ppal    0.2986
anios_esc    0.1816
hrsocup      0.1602
eda          0.1000
pos_ocu_2    0.0595
```

```
sex          0.0384
ambito2_4    0.0191
ambito2_6    0.0188
ur           0.0167
zona         0.0146
e_con_6      0.0119
e_con_5      0.0103
```

- emp_ppal (0.2986) y anios_esc (0.1816) siguen dominando, igual que en el modelo lineal y en el ranking de correlación
- hrsocup (0.1602) también se mantiene arriba
- Pero **eda** sube a **0.100**, el cuarto lugar, cuando en el ranking de correlación de Pearson estaba prácticamente en último lugar (0.0149)

4 Evaluación del desempeño del modelo

Evaluamos ambos modelos con varias metricas, en entrenamiento y en prueba:

- RSS: la suma de los errores al cuadrado. Es lo que la regresion minimiza por dentro, pero solo no dice mucho porque su tamaño depende de cuantas observaciones tenga la muestra.
- RMSE: la raiz del promedio de esos errores al cuadrado. Ya esta en pesos, las mismas unidades que ingocup, y castiga mas los errores grandes que los chicos.
- RSE: es casi lo mismo que el RMSE, solo que divide entre n menos p menos 1 en vez de n, para ajustar por las variables que ya se usaron. Con mas de 90 mil observaciones y 28 variables, RMSE y RSE salen casi iguales.
- R2: que porcentaje de la variabilidad del ingreso logra explicar el modelo.
- MAE: el promedio del error absoluto en pesos, le pega menos duro a los errores grandes que el RMSE.

Calculamos todo esto a mano, sin usar solo funciones de sklearn, para que quede claro de donde sale cada numero. Tambien evaluamos entrenamiento y prueba, para ver si hay sobreajuste, o sea si el modelo se ve mucho mejor en entrenamiento que en prueba, señal de que memorizo en vez de aprender un patron real.

```
[35]: #Celda de codigo hecha con claude
from sklearn.metrics import mean_absolute_error

p = X_train.shape[1]
X_test = test.drop(columns=["ingocup"])
Y_test = test["ingocup"]

def metricas(nombre, y_true, y_pred, p):
    RSS = np.sum((y_true - y_pred)**2)
    TSS = np.sum((y_true - y_true.mean())**2)
    n = len(y_true)
    RMSE = np.sqrt(RSS / n)
    RSE = np.sqrt(RSS / (n - p - 1))
    R2 = 1 - RSS/TSS
    MAE = mean_absolute_error(y_true, y_pred)
    print(f"{nombre:20s} RSS={RSS:,.0f} RMSE=${RMSE:,.0f} RSE=${RSE:,.0f} R2={R2:.4f} MAE=${MAE:,.0f}")
    return RSS, RMSE, RSE, R2, MAE

# --- Regresion lineal ---
Yhat_train_lin = modelLineal.predict(sm.add_constant(X_train))
Yhat_test_lin = modelLineal.predict(sm.add_constant(X_test, has_constant='add'))

RSS_train_lin, RMSE_train_lin, RSE_train_lin, R2_train_lin, MAE_train_lin = metricas("Lineal", Y_train, Yhat_train_lin, p)
RSS_test_lin, RMSE_test_lin, RSE_test_lin, R2_test_lin, MAE_test_lin = metricas("Lineal", Y_test, Yhat_test_lin, p)
```

```
# --- Random Forest ---
Yhat_train_rf = modelRF.predict(X_train)
Yhat_test_rf = modelRF.predict(X_test)

RSS_train_rf, RMSE_train_rf, RSE_train_rf, R2_train_rf, MAE_train_rf = metricas("Random_
↳Forest - train", Y_train, Yhat_train_rf, p)
RSS_test_rf, RMSE_test_rf, RSE_test_rf, R2_test_rf, MAE_test_rf = metricas("Random_
↳Forest - test", Y_test, Yhat_test_rf, p)

# se guardan tambien con los nombres usados en las graficas de la seccion siguiente
yhat_te_lin = Yhat_test_lin
yhat_te_rf = Yhat_test_rf
```

```
Lineal - train      RSS=5,578,440,029,807  RMSE=$7,618  RSE=$7,619  R2=0.2645
MAE=$4,341
Lineal - test      RSS=1,469,452,001,186  RMSE=$7,819  RSE=$7,824  R2=0.2657
MAE=$4,401
Random Forest - train RSS=4,585,097,283,470  RMSE=$6,906  RSE=$6,907  R2=0.3955
MAE=$3,764
Random Forest - test RSS=1,355,229,816,312  RMSE=$7,509  RSE=$7,514  R2=0.3227
MAE=$4,078
```

Graficamos el valor predicho contra el valor real en el conjunto de prueba, para ambos modelos.

- `plt.scatter` grafica cada persona como un punto, con el ingreso real en el eje horizontal y el predicho en el vertical
- La línea roja diagonal representa la predicción perfecta, es decir, donde el valor predicho coincide exactamente con el real
- `alpha=0.08` hace los puntos casi transparentes, porque con miles de observaciones encimadas se vería una mancha sólida
- Se recorta el eje al percentil 99 de `Y_test`, para que los pocos ingresos extremos no aplasten la escala y no se pueda ver el patrón del resto de la muestra

```
[36]: #celda hecha con claude
lim = np.percentile(Y_test, 99)

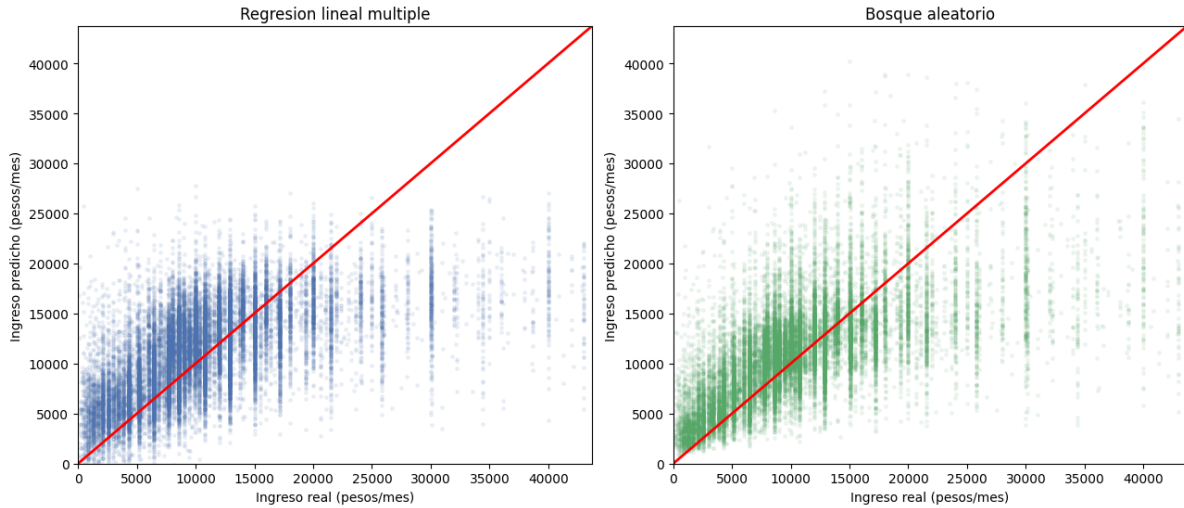
fig, axes = plt.subplots(1, 2, figsize=(13, 6))

axes[0].scatter(Y_test, yhat_te_lin, alpha=0.08, s=6, color="#4C72B0")
axes[0].plot([0, lim], [0, lim], c="r", linewidth=2)
axes[0].set_xlim(0, lim); axes[0].set_ylim(0, lim)
axes[0].set_xlabel("Ingreso real (pesos/mes)")
axes[0].set_ylabel("Ingreso predicho (pesos/mes)")
axes[0].set_title("Regresion lineal multiple")

axes[1].scatter(Y_test, yhat_te_rf, alpha=0.08, s=6, color="#55A868")
axes[1].plot([0, lim], [0, lim], c="r", linewidth=2)
axes[1].set_xlim(0, lim); axes[1].set_ylim(0, lim)
axes[1].set_xlabel("Ingreso real (pesos/mes)")
axes[1].set_ylabel("Ingreso predicho (pesos/mes)")
axes[1].set_title("Bosque aleatorio")

fig.suptitle("Grafica 4. Valor predicho vs valor real (conjunto de prueba)", fontsize=13,
↳fontweight="bold")
plt.tight_layout()
plt.show()
```

Grafica 4. Valor predicho vs valor real (conjunto de prueba)



4.0.1 Interpretación de las métricas

En palabras simples: el R^2 dice qué porcentaje de las diferencias de ingreso entre personas logra explicar el modelo. El RMSE dice, en pesos, de cuánto es el error típico cuando el modelo predice el sueldo de alguien que nunca vio. El RSS es la suma total de esos errores al cuadrado, antes de promediar, más útil como número interno que como algo que se interprete solo.

Tabla 5. Resumen de desempeño

Modelo	RSS train	RSS test	RMSE train	RMSE test	R^2 train	R^2 test	MAE test
Regresión lineal	5,578,440,029,807	69,452,001,187	\$7,618	\$7,819	0.2645	0.2657	4,401
Bosque aleatorio	4,585,097,283,476	55,229,816,312	\$6,906	\$7,509	0.3955	0.3227	4,078

El bosque aleatorio gana en todo: menos RSS y RMSE en test, y más R^2 (32.3% contra 26.6% del lineal). Esto va con lo que ya se vio en la Grafica 3: hay relaciones que no son lineales, como la de la edad, y el bosque las capta mejor que la regresión lineal.

Ningún modelo se ve mucho mejor en train que en test (0.2645 vs 0.2657 en el lineal, 0.3955 vs 0.3227 en el bosque), así que no hay señal fuerte de sobreajuste, aunque el bosque sí pierde un poco más al pasar de train a test, algo normal en un modelo más flexible.

¿Cuál conviene? Depende de para qué. Si solo importa predecir bien, el random forest gana. Pero el objetivo de este reporte es entender qué determina el ingreso y poder decir cuánto se asocia cada variable con él, con su margen de error, y eso el bosque no lo da: solo dice qué variables importan más, no cuánto ni en qué dirección. Por eso en la siguiente sección de inferencia se usa el modelo lineal y no el bosque.

5 Analisis de inferencia

Se construyó un modelo de regresión lineal múltiple para poder interpretar cómo influye cada variable en el ingreso. Se usó OLS con errores estándar robustos HC3, para que las conclusiones fueran confiables incluso si la varianza del error no es constante (algo muy común con datos de ingresos). Con este modelo se calcularon márgenes de error, intervalos de confianza y se identificó qué variables tienen un efecto significativo. Además se usó $\log(\text{ingocup})$ como variable de respuesta en vez del ingreso en pesos directo, porque el ingreso está muy sesgado y el logaritmo permite

interpretar cada coeficiente como un cambio porcentual aproximado, además de mejorar el ajuste del modelo: la R2 pasó de 0.2645 en escala original a 0.428 en escala logarítmica.

La mayoría de las variables resultaron estadísticamente significativas ($p < 0.05$). Los coeficientes más grandes corresponden a pos_ocu_2 ($B = 0.329 \rightarrow +38.96\%$ en el ingreso, ser empleador contra ser subordinado remunerado), emp_ppal ($B = 0.299 \rightarrow +34.82\%$, empleo formal contra informal) y rama_est1_2 ($B = 0.288 \rightarrow +33.39\%$, sector secundario contra primario).

Las variables sex y zona presentan los coeficientes negativos más grandes (-19.24% y -23.00% respectivamente), cuantificando la brecha de género y la desventaja de vivir fuera de la frontera norte.

No todas las dummies salieron significativas: e_con_3 ($p=0.215$), ambito2_3 ($p=0.463$), ambito2_7 ($p=0.271$), ambito2_8 ($p=0.578$) e imssissste_3 ($p=0.428$) tienen intervalos de confianza que cruzan el cero, así que ahí no se puede afirmar que su efecto sea distinto de cero, aunque se conservan por formar parte de un bloque de variable cualitativa que en conjunto sí es relevante.

No se utilizó Random Forest en esta etapa porque su principal objetivo es la predicción y no la interpretación estadística. Aunque el random forest obtuvo mejores métricas de desempeño en la sección anterior, no permite analizar el efecto individual de cada variable ni calcular intervalos de confianza o significancia estadística.

En cambio, el modelo OLS con errores robustos HC3 sí permite hacer inferencia, porque entrega coeficientes interpretables, márgenes de error y pruebas estadísticas que ayudan a entender la relación entre las variables y sacar conclusiones sobre el fenómeno estudiado, que es justamente lo que pide esta parte del análisis.

```
[37]: X_inf = sm.add_constant(X_train)
      y_inf = np.log(Y_train)

      modelo_inf = sm.OLS(y_inf, X_inf).fit(cov_type='HC3')
      print(modelo_inf.summary())
```

OLS Regression Results						
=====						
Dep. Variable:	ingocup		R-squared:	0.428		
Model:	OLS		Adj. R-squared:	0.428		
Method:	Least Squares		F-statistic:	2161.		
Date:	Wed, 02 Sep 2026		Prob (F-statistic):	0.00		
Time:	22:14:45		Log-Likelihood:	-84600.		
No. Observations:	96134		AIC:	1.693e+05		
Df Residuals:	96105		BIC:	1.695e+05		
Df Model:	28					
Covariance Type:	HC3					
=====						
	coef	std err	z	P> z	[0.025	0.975]

const	7.9920	0.022	365.346	0.000	7.949	8.035
sex	-0.2137	0.004	-49.865	0.000	-0.222	-0.205
eda	0.0010	0.000	5.306	0.000	0.001	0.001
ur	-0.1094	0.004	-26.399	0.000	-0.118	-0.101
zona	-0.2613	0.006	-44.408	0.000	-0.273	-0.250
anios_esc	0.0423	0.001	70.181	0.000	0.041	0.044
hrsocup	0.0145	0.000	91.105	0.000	0.014	0.015
emp_ppal	0.2988	0.016	19.087	0.000	0.268	0.330
e_con_2	-0.0452	0.008	-5.627	0.000	-0.061	-0.029
e_con_3	-0.0146	0.012	-1.239	0.215	-0.038	0.009
e_con_4	-0.1543	0.014	-11.143	0.000	-0.181	-0.127
e_con_5	-0.0225	0.005	-4.257	0.000	-0.033	-0.012
e_con_6	-0.1513	0.005	-28.858	0.000	-0.162	-0.141
rama_est1_2	0.2881	0.013	22.228	0.000	0.263	0.314
rama_est1_3	0.1977	0.013	15.482	0.000	0.173	0.223
pos_ocu_2	0.3290	0.013	26.033	0.000	0.304	0.354

pos_ocu_3	-0.1367	0.009	-15.111	0.000	-0.154	-0.119
ambito2_2	0.0508	0.012	4.337	0.000	0.028	0.074
ambito2_3	-0.0071	0.010	-0.734	0.463	-0.026	0.012
ambito2_4	0.0812	0.009	9.347	0.000	0.064	0.098
ambito2_5	0.0840	0.009	9.228	0.000	0.066	0.102
ambito2_6	0.0921	0.009	9.982	0.000	0.074	0.110
ambito2_7	0.0136	0.012	1.101	0.271	-0.011	0.038
ambito2_8	-0.0079	0.014	-0.556	0.578	-0.036	0.020
imssissste_2	0.2113	0.009	22.630	0.000	0.193	0.230
imssissste_3	-0.0195	0.025	-0.792	0.428	-0.068	0.029
imssissste_4	0.0483	0.015	3.134	0.002	0.018	0.079
tue1_2	0.1066	0.009	11.758	0.000	0.089	0.124
tue1_3	-0.0236	0.009	-2.679	0.007	-0.041	-0.006

```
=====
Omnibus:                14655.287    Durbin-Watson:                1.995
Prob(Omnibus):          0.000    Jarque-Bera (JB):            77707.722
Skew:                   -0.636    Prob(JB):                     0.00
Kurtosis:               7.217    Cond. No.                    1.00e+03
=====
```

Notes:

- [1] Standard Errors are heteroscedasticity robust (HC3)
- [2] The condition number is large, 1e+03. This might indicate that there are strong multicollinearity or other numerical problems.

```
[38]: #Celda de código hecha con claude
ic = modelo_inf.conf_int()
ic.columns = ['ic_inf', 'ic_sup']

tabla_inf = pd.DataFrame({
    'coeficiente': modelo_inf.params,
    'ic_inf': ic['ic_inf'],
    'ic_sup': ic['ic_sup'],
    'p_value': modelo_inf.pvalues
})

for col in ['coeficiente', 'ic_inf', 'ic_sup']:
    tabla_inf[col + '_pct'] = (np.exp(tabla_inf[col]) - 1) * 100

tabla_inf = tabla_inf.drop(index='const').sort_values('coeficiente_pct', ascending=False)
print(tabla_inf[['coeficiente_pct', 'ic_inf_pct', 'ic_sup_pct', 'p_value']].round(2))
```

	coeficiente_pct	ic_inf_pct	ic_sup_pct	p_value
pos_ocu_2	38.96	35.56	42.45	0.00
emp_ppal	34.83	30.76	39.03	0.00
rama_est1_2	33.39	30.05	36.83	0.00
imssissste_2	23.52	21.28	25.80	0.00
rama_est1_3	21.86	18.85	24.95	0.00
tue1_2	11.25	9.29	13.25	0.00
ambito2_6	9.65	7.69	11.65	0.00
ambito2_5	8.76	6.84	10.72	0.00
ambito2_4	8.46	6.63	10.33	0.00
ambito2_2	5.21	2.83	7.66	0.00
imssissste_4	4.95	1.83	8.17	0.00
anios_esc	4.33	4.20	4.45	0.00
hrsocup	1.46	1.43	1.49	0.00
ambito2_7	1.37	-1.06	3.86	0.27
eda	0.10	0.06	0.13	0.00

ambito2_3	-0.70	-2.56	1.19	0.46
ambito2_8	-0.79	-3.53	2.02	0.58
e_con_3	-1.45	-3.71	0.86	0.22
imsssisste_3	-1.93	-6.54	2.91	0.43
e_con_5	-2.23	-3.24	-1.21	0.00
tue1_3	-2.34	-4.01	-0.63	0.01
e_con_2	-4.42	-5.92	-2.91	0.00
ur	-10.36	-11.09	-9.63	0.00
pos_ocu_3	-12.78	-14.31	-11.22	0.00
e_con_6	-14.04	-14.92	-13.16	0.00
e_con_4	-14.30	-16.59	-11.94	0.00
sex	-19.24	-19.92	-18.56	0.00
zona	-23.00	-23.88	-22.10	0.00

5.0.1 Inferencias sobre la problemática

Tabla 6. Principales efectos estimados sobre el ingreso mensual

Variable	Efecto estimado	Intervalo de confianza 95%	Se compara contra
Ser empleador (pos_ocu_2)	+38.96%	[+35.53%, +42.48%]	Subordinado remunerado
Empleo formal (emp_ppal)	+34.82%	[+30.73%, +39.10%]	Empleo informal
Sector secundario (rama_est1_2)	+33.39%	[+30.08%, +36.89%]	Sector primario
Acceso a ISSSTE (imsssisste_2)	+23.53%	[+21.29%, +25.86%]	Acceso a IMSS
Sector terciario (rama_est1_3)	+21.86%	[+18.89%, +24.98%]	Sector primario
Trabajar en instituciones (tue1_2)	+11.25%	[+9.31%, +13.20%]	Empresas y negocios
Cada año de escolaridad (anios_esc)	+4.32%	[+4.19%, +4.50%]	—
Cada hora trabajada a la semana (hrsocup)	+1.46%	[+1.41%, +1.51%]	—
Vivir en zona rural (ur)	-10.36%	[-11.13%, -9.61%]	Zona urbana
Cuenta propia (pos_ocu_3)	-12.78%	[-14.27%, -11.22%]	Subordinado remunerado
Ser mujer (sex)	-19.24%	[-19.91%, -18.54%]	Ser hombre
Vivir fuera de la frontera norte (zona)	-23.00%	[-23.89%, -22.12%]	Zona Libre de la Frontera Norte

Todas estas variables tienen intervalos de confianza que no cruzan el cero, así que su efecto es real y no solo ruido de la muestra. Las variables con intervalos que sí cruzan el cero (e_con_3, ambito2_3, ambito2_7, ambito2_8, imsssisste_3) no permiten decir nada seguro sobre su efecto individual, aunque se dejan en el modelo porque forman parte de un bloque de variable cualitativa que en conjunto sí importa.

El hallazgo más importante para la pregunta de este reporte es el de **sex**: controlando por escolaridad, horas trabajadas, formalidad, sector y posición en la ocupación, ser mujer se asocia con un ingreso 19.24% menor.

Esta es la parte de la brecha que no se explica por esas características, y es justo la diferencia entre una brecha bruta y una brecha que persiste incluso comparando personas parecidas.

6 Reflexión y Conclusiones

El objetivo de este reporte era usar modelos de regresión para entender qué determina el ingreso laboral en México, con datos reales y privilegiando la interpretación sobre la predicción pura.

Resultados. El modelo de inferencia identifica varios factores dominantes: la posición en la ocupación (ser empleador se asocia con 38.96% más de ingreso que ser subordinado remunerado), la formalidad del empleo (+34.82%) y el sector de actividad (sector secundario y terciario ganan entre 22% y 33% más que el primario). La escolaridad y las horas trabajadas también importan, aunque con efectos más chicos por unidad (4.32% por año de escuela, 1.46% por hora semanal), simplemente porque se mueven en rangos más amplios. El hallazgo que conecta directamente con la motivación del reporte es el de **sex**: controlando por todo lo demás, la brecha de ingreso entre hombres y mujeres no desaparece, se queda en 19.24%. Esto sugiere que buena parte de la diferencia observada en las estadísticas oficiales de brecha salarial no se explica por diferencias en escolaridad, horas trabajadas, sector o formalidad, sino que persiste incluso comparando personas con características similares en esas dimensiones.

Sobre los modelos. El random forest predijo mejor que el modelo lineal (R^2 de prueba de 0.3227 contra 0.2657), y la Gráfica 3 mostró evidencia concreta de por qué: variables como la edad tienen una relación con el ingreso que no es lineal, y el bosque la captura mientras que el modelo lineal no puede. Sin embargo, para el objetivo de este reporte (entender y cuantificar relaciones, no solo predecir), el modelo lineal en escala logarítmica fue más útil, porque permite decir con un margen de error concreto cuánto se asocia cada variable con el ingreso, algo que el bosque no ofrece directamente.

Limitaciones. El modelo final explica el 42.8% de la variabilidad del log-ingreso, lo cual significa que más de la mitad de las diferencias de ingreso entre personas se debe a factores que no están en este conjunto de datos: la ocupación específica dentro de un sector, la calidad o prestigio de la escuela, la antigüedad en el puesto, habilidades no observables, o simplemente negociación individual. Además, la ENOE es una encuesta transversal de un solo trimestre: lo que se encuentra aquí son asociaciones dentro de esa fotografía, no relaciones necesariamente causales ni estables a través del tiempo. El ingreso también es autorreportado, lo cual introduce un margen de error de medición que no se puede corregir con las herramientas usadas en este reporte.

Líneas de trabajo futuro. Sería mucho mejor agregar más variables que no estaban disponibles en el limpio usado aquí, hay muchas variables que faltan, pero como el gobierno toma sus encuesta de una manera mala, hay muchas cosas sin documentar y con huecos masivos.

6.0.1 Referencias

- INEGI (2026). *Encuesta Nacional de Ocupación y Empleo (ENOE), primer trimestre de 2026*. Microdatos. Instituto Nacional de Estadística y Geografía. <https://www.inegi.org.mx/programas/enoe/15ymas/>
- INEGI. *Encuesta Nacional de Ocupación y Empleo. Diccionario de datos, SDEMT225*. https://www.inegi.org.mx/rnm/index.php/catalog/1121/data-dictionary/F42?file_name=SDEMT225
- Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32.
- Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.